## 前置知識與目標

- 前置知識為陣列、堆積、遞迴、機率期望與第 1 章複雜度定義。
- 能說明每種排序的不變量、正確性與成本來源，區分穩定性、原地性及比較模型。
- 能根據已排序程度、鍵域、記憶體限制與最壞時間需求選擇方法。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 排序契約與穩定性

- 輸出須為輸入的排列，並滿足 $A[i] \le A[i+1]$。只檢查有序不足以保證沒有遺失或複製元素。
- 穩定排序保留等鍵記錄的原相對順序。例如 $(2,A),(1,X),(2,B)$ 排完仍須讓 A 在 B 前。
- 本教材稱「原地」為只需 $O(1)$ 額外資料陣列空間，遞迴堆疊另列，不把它隱藏。
- 比較器必須符合一致的順序關係。C++ 排序要求 strict weak ordering，隨意使用 $\le$ 作比較器會違反要求。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 插入排序的不變量

1. 在第 i 輪前，`A[0,i)` 已有序，且為原前 i 個元素的排列。
2. 保存 `key=A[i]`，將前綴中大於 key 的值向右移，直到插入位置空出。
3. 把 key 寫入該位置，得到有序前綴 `A[0,i+1)`。

只搬動「嚴格大於」key 的元素即可保留穩定性。原例 `[5,2,4,2]` 依序成 `[2,5,4,2]`、`[2,4,5,2]`、`[2,2,4,5]`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 插入排序與 inversions

- inversion（逆序對）為 `i<j` 且 `A[i]>A[j]` 的配對。每次右移消去目前 key 對應的一個逆序。
- 若總逆序數為 I，標準插入排序的時間可寫 `Θ(n+I)`。
- 已有序時 `I=0`，最佳 `Θ(n)`。完全反序且鍵互異時 `I=n(n-1)/2`，最壞 `Θ(n²)`。
- 這解釋插入排序適合小型或接近有序資料，並非只靠「平均很快」的模糊印象。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 快速排序的 partition 契約

- 本頁使用三向分割，選 pivot 值 p，將區間分成 `<p`、`=p`、`>p`，只遞迴左右兩區。
- 掃描期間維持 `[l,lt)<p`、`[lt,i)=p`、`[i,gt)` 未分類、`[gt,r)>p`。
- 小於 p 就交換至 lt 並前移 lt、i；等於 p 前移 i；大於 p 就減少 gt 並交換，i 留下重查。
- 終止量 `gt-i` 每輪下降，終止時沒有未分類元素。分割成本 `Θ(r-l)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 三向分割逐輪索引與遞迴契約

```
partition(A,l,r,p):
    lt=l; i=l; gt=r
    while i<gt:
        if A[i]<p: swap(A[lt],A[i]); lt+=1; i+=1
        else if A[i]>p: gt-=1; swap(A[i],A[gt])
        else: i+=1
    return (lt,gt)
quicksort(A,l,r):
    if r-l<2: return
    choose p from A[l:r]
    (lt,gt)=partition(A,l,r,p)
    quicksort(A,l,lt); quicksort(A,gt,r)
```

對[4,2,4,7,1,4]，狀態(A,lt,i,gt)為：初始(原列,0,0,6)；讀4→(原列,0,1,6)；讀2→([2,4,4,7,1,4],1,2,6)；讀4→(同列,1,3,6)；讀7→([2,4,4,4,1,7],1,3,5)；重讀4→(同列,1,4,5)；讀1→([2,1,4,4,4,7],2,5,5)。只有[0,2)及[5,6)遞迴。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 三向分割逐輪索引與遞迴契約 （續）

等值區必非空，因pivot取自該區間，確保遞迴子區變小。若pivot不在資料且選到空等值區，某一側可能等於原區間而不終止。比較次數界須說明隨機選pivot；固定中點值不等於隨機選擇。較小側遞迴／較大側迴圈只保證堆疊O(log n)，不改最壞Θ($n^2$)。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 快排的最壞與期望

- 原例 `[4,2,4,7,1,4]` 選 p=4，可分成 `[2,1]` | `[4,4,4]` | `[7]`，區內順序不保證。
- 互異鍵若每次選到極端值，`T(n)=T(n-1)+Θ(n)`，最壞 `Θ(n²)`。
- 對固定互異輸入，每個子問題均勻隨機選 pivot，期望時間為 `Θ(n log n)`。若採經典「pivot 與每個其他鍵恰比較一次」的計數模型，期望比較數才是 `2(n+1)H_n−4n`。期望來自內部亂數。
- 三向分割可一次消去大量等鍵。此互異鍵的精確比較數公式不能直接套到所有三向實作。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 隨機快排的比較期望與堆疊推導

```
quicksort_small_side(A,l,r):
    while r-l>=2:
        p=uniformly_random_value_from(A[l:r])
        (lt,gt)=partition(A,l,r,p)
        if lt-l<r-gt:
            quicksort_small_side(A,l,lt); l=gt
        else:
            quicksort_small_side(A,gt,r); r=lt
```

互異鍵排序後以排名 $1,\dots,n$ 表示，令 $X_{ij}$ 指示排名 $i,j$ 是否曾直接比較。經典二向版本每輪把 pivot 與其餘每鍵恰比較一次；兩鍵被比較若且唯若排名區間 $[i,j]$ 第一個被選作 pivot 者是兩端之一。區間內 $j-i+1$ 個鍵對稱，故 $\Pr(X_{ij}=1)=2/(j-i+1)$。線性期望給 $\mathbb{E}[C]=\sum_{i<j}2/(j-i+1)=2\sum_{d=1}^{n-1}(n-d)/(d+1)=2(n+1)H_n-4n=\Theta(n\log n)$，其中 $H_n=\sum_{k=1}^{n}1/k$。



## 隨機快排的比較期望與堆疊推導 （續）

三向分割本稿以兩次順序測試判別<、>、=，互異鍵時每輪仍對每個非pivot鍵做有界常數次比較，因此保留期望O(n log n)，但不能原封使用上述精確比較數。每個pivot對每個非pivot鍵至少要比較一次，相同的隨機排名分割也給上述指示變量和的下界，故為期望Θ(n log n)；這裡限互異鍵。全相同鍵三向分割一次結束，僅Θ(n)。

空間推導與時間期望分開：上方程序只真正遞迴較小非等值區，其長度≤(r−l−1)/2，活躍堆疊的子問題大小每次至少減半，故最壞O(log n)框架。較大區改由同一框架的while處理。即使亂數一直選到極端鍵，掃描總長n+(n−1)+⋯仍為Θ(n²)，只是堆疊不再線性。

## 遞迴空間與深度保護

- 直接遞迴兩側的快排，最壞堆疊深度 `Θ(n)`。資料分割原地不代表整體額外空間必為常數。
- 每次只遞迴較小的一側，較大的一側改迴圈，可把堆疊限制為 `O(log n)`，因遞迴子問題至多減半。
- 此改寫改善空間，不會消除 `Θ(n²)` 的最壞比較成本。
- introspective sorting 另在遞迴過深時切換到 heapsort。這是工程延伸，不宣稱原書必然包含。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 合併排序與穩定合併

- 左右兩段已排序時，比較各自尚未取出最小值，取較小者輸出。等鍵時先取左段即可保留穩定性。
- 不變量為輸出是已消耗輸入的有序合併，而且輸出末值不大於兩段未消耗元素。
- 原例 `[2_A,5]` 與 `[2_B,4]` 合併為 `[2_A,2_B,4,5]`。若等鍵先取右段，會破壞原順序。
- 陣列標準合併用 `Θ(n)` 緩衝區。遞迴 `T(n)=2T(n/2)+Θ(n)`，最壞 `Θ(n log n)`。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 穩定合併與 merge-sort 的完整程序

```
merge(L,R):
    i=0; j=0; out=[]
    while i<len(L) and j<len(R):
        if L[i].key<=R[j].key: out.append(L[i]); i+=1
        else: out.append(R[j]); j+=1
    append remaining L[i:]; append remaining R[j:]
    return out
merge_sort(A):
    if len(A)<2: return A
    m=len(A)//2
    return merge(merge_sort(A[:m]),merge_sort(A[m:]))
```

合併L=[2_A,5]、R=[2_B,4]：先等鍵取A，游標(1,0)，輸出[A]；再取B，(1,1)，輸出[A,B]；再取4，(1,2)，輸出[A,B,4]；補5結束。每次輸出兩段尚未消耗最小值中較小者，證明排序；每個輸入只消耗一次，證明排列保持。

此 merge_sort 的切片版是語義偽碼；共享 buffer 的 C++ 實作避免每層保留不必要副本。



## 堆積排序的有序後綴

- 遞增排序先把陣列建成 max-heap，再交換根與末端，把末端排除在堆範圍外。
- 不變量為前段是 max-heap，後段已遞增，且前段所有鍵不大於後段所有鍵。
- 每次取出的根是尚未排序資料的最大值，放到後段最前位置。對剩餘根 sift-down 復原堆序。
- 建堆 `O(n)`，後續 n 次下沉總最壞 `O(n log n)`。可迭代實作為 `O(1)` 額外空間，通常不穩定。

## 堆積排序的完整驅動與有序後綴追蹤

```
heap_sort(A):
    build_max_heap(A)
    for end=len(A)-1 down to 1:
        swap(A[0],A[end])
        sift_down_max(A,0,end)    // heap is [0,end), sorted suffix excluded
```

堆積排序同一鍵列先成max-heap [5,2,4,2]。取5後[2,2,4|5]，下沉為[4,2,2|5]；取4後[2,2|4,5]；取2後[2|2,4,5]。豎線右邊是固定有序後綴，不能再參與sift。此非穩定算法即使值結果相同，也不能據此聲稱記錄身分順序保持。

sift_down_max 使用兩子中較大者，停止於 parent≥該子；索引與第 5 章 min 版相同，不等號相反。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 計數排序的鍵域假設

- 鍵是 `[0,K)` 的整數時，以 K 格計數。前綴和給出各鍵在輸出中的結束位置。
- 由右向左掃原記錄，把 key=k 的記錄放入 `--end[k]`，可保留等鍵原次序。
- 時間 `Θ(n+K)`，輔助計數與輸出共 `Θ(n+K)`。K 遠大於 n 時可能不划算。
- 計數排序讀取鍵值並做直接索引，不只比較兩鍵，因此不受純比較排序下界直接限制。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 穩定計數排序的配置與逐筆散佈

```
stable_count(records,K):
    out=array(len(records)); count=[0]*K
    for x in records: require 0<=x.key<K; count[x.key]+=1
    end=prefix_sum_inclusive(count)
    for x in reverse(records): end[x.key]-=1; out[end[x.key]]=x
    return out
```

對[(5,A),(2,B),(4,C),(2,D)]、K=6，count=[0,0,2,0,1,1]、end=[0,0,2,2,3,4]。由右掃D→out[1]，C→out[2]，B→out[0]，A→out[3]；得[B,D,C,A]，B仍在D前。LSD每輪套此穩定程序，不能把不穩定交換版當成等價。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## LSD 基數排序的穩定性需求

- 用 b 為基底，鍵有 d 位。由最低位到最高位，每輪對目前位做穩定排序。
- 歸納不變量為第 k 輪後，資料依最低 k 位的整數值排序。下一輪穩定性保留相同高位內的低位順序。
- 原例 `31,14,22,11`，個位排序為 `31,11,22,14`，十位穩定排序後為 `11,14,22,31`。
- 用穩定計數排序，每輪 `O(n+b)`，共 `O(d(n+b))`。有號數、變長字串與 Unicode 需要另定鍵編碼及順序。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## LSD 的完整驅動與逐位不變量

```
LSD(A,b,d):
    for position=0..d-1:
        A=stable_count_by_digit(A,position,b)
    return A
```

LSD 固定寬 d 位、基底 b≥2，digit=(key//b^position) mod b；鍵須為非負整數或先作保序編碼。

31,14,22,11第一輪digit為1,4,2,1，穩定次序31,11,22,14；第二輪digit為3,1,2,1，穩定輸出11,14,22,31。第k輪後按低k位排序的不變量由等高位的穩定保留推出。若讀取一位需任意精度除法，Θ(d(n+b))的常數digit前提失效。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 比較排序下界的決策樹

- 假設 n 個互異鍵，演算法只能用兩鍵比較獲取順序資訊。確定性算法的比較歷程形成二元決策樹。
- n! 個輸入排列需要至少 n! 個可區別葉節點。高度 h 的二元樹至多 2^h 葉，因此 $h \ge \lceil \log_2(n!) \rceil$。
- 因 $n! \ge (n/2)^{n/2}$，有 $\log_2(n!) = \Omega(n \log n)$。所以最壞比較次數至少 $\Omega(n \log n)$。
- 限制在比較模型、互異鍵及一般排列。此證明沒有禁止已知鍵域的線性排序，也沒有聲稱每個輸入都需 n log n 次比較。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

## 方法比較必須保留限定

| 方法 | 最佳 | 最壞 | 主要額外空間 | 穩定 |
| --- | --- | --- | --- | --- |
| 插入 | $\Theta(n)$ | $\Theta(n^2)$ | $O(1)$ | 是，僅移動嚴格較大值 |
| 隨機快排 | 視分割 | $\Theta(n^2)$，期望 $\Theta(n \log n)$ | 堆疊可 $O(\log n)$ | 通常否 |
| 合併 | $\Theta(n \log n)$，此標準版 | $\Theta(n \log n)$ | $\Theta(n)$ | 是，等鍵取左 |
| 堆積 | 視鍵重複與實作 | $\Theta(n \log n)$ | $O(1)$ 迭代版 | 否 |
| LSD 基數 | $\Theta(d(n+b))$，此版本 | 同左 | $\Theta(n+b)$ | 每輪需穩定 |

表列成本以常數時間鍵操作為前提，不用不相容的模型比較「快慢」。



## 常見錯誤與驗證

- 測試排序必須同時檢查順序、元素多重集合與記錄穩定性。只有 `is_sorted` 可能漏掉資料遺失。
- 全相同鍵、已排序、反序、空陣列、單元素及大量重複鍵，會暴露不同分割邊界問題。
- `examples/sorting.cpp` 用固定種子資料與標準排序結果比對原創插入、合併及三向快排，並檢查負數與重複鍵。
- 程式案例不是正確性證明。分割不變量和遞迴終止論據仍需保留。

## 來源與延伸

- 排序下界與非比較排序課程定位核對 MIT 6.006 2008 L10–L11。
- 比較器要求核對 C++ algorithms requirements。C++ 標準介面要求不等於標準規定某一個固定實作算法。
- 逆序數、分割追蹤、資料與練習均為原創教學編排。introsort 為延伸，不宣稱對應原書第二版節次。

Lin Cing
https://github.com/xinhi6716/computer-science-self-study



## 參考資料與署名

Ellis Horowitz, Sartaj Sahni, Dinesh Mehta. Fundamentals of Data Structures in C++, 2nd Edition.

Lin Cing 原創教學整理。原創內容以 CC BY 4.0 授權；參考書籍不隨教材散布。
https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study